Right Smaller Than

AlgoExpert · Very Hard · Binary Search Trees

The problem

- For each element, how many elements to its right are strictly smaller?
- Insert from the right into an augmented BST tracking left-subtree sizes.

Intuition

- Insert from the right into a BST that knows its left-subtree sizes; each insert counts the smaller values already there.

Approach

- 1. Iterate from the end
- 2. Insert into an augmented BST
- 3. Add left sizes when going right
- 4. Record the count

Java solution (1/2)

```java
public class RightSmallerThan {
    private static class Node {
        int value, leftSize;
        Node left, right;
        Node(int v) { value = v; }
    }
```

Java solution (2/2)

```java
public static List<Integer> rightSmallerThan(List<Integer> array) {
    int n = array.size();
    Integer[] out = new Integer[n];
    if (n == 0) return new ArrayList<>();
    Node root = new Node(array.get(n - 1));
    out[n - 1] = 0;
    for (int i = n - 2; i >= 0; i--) {
        int v = array.get(i), smaller = 0;
        Node cur = root;
        while (true) {
            if (v < cur.value) {
                cur.leftSize++;
                if (cur.left == null) { cur.left = new Node(v); break; }
                cur = cur.left;
            } else {
                smaller += cur.leftSize + (v > cur.value ? 1 : 0);
                if (cur.right == null) { cur.right = new Node(v); break; }
                cur = cur.right;
            }
        }
        out[i] = smaller;
    }
    return new ArrayList<>(List.of(out));
}
```

Complexity

- Time: O(n log n) avg, O(n^2) worst
- Space: O(n)

Recap

- Insert from the right into a BST that knows its left-subtree sizes; each insert counts the smaller values already there.
- Complexity: O(n log n) avg, O(n^2) worst